

Multi-Platform Container Rebuild Report

Section 1: Overview

What was rebuilt and why

This project runs the NCAR CESM (Community Earth System Model) climate modeling framework inside a Docker container. Researchers pull the container image, launch it, and open JupyterLab in their browser to run analysis notebooks against CESM simulation data. The container packages everything needed: an operating system, scientific Fortran/C libraries, the CESM source code and build system, a full Python scientific stack (NumPy, xarray, cartopy, etc.), and JupyterLab itself.

The original container image (`escomp/cesm-lab-neon`) was built three years ago on CentOS 8 (which lost security support in December 2021) and only supported Intel/AMD processors. It shipped Python 3.7 (end-of-life since June 2023) and hundreds of outdated packages. Most critically, it could not run natively on Apple Silicon Macs (M1/M2/M3/M4), which use a different processor architecture called ARM64. Team members on these machines had to run the container through a translation layer that was 2–3x slower and crashed frequently during computationally intensive operations like cartopy map rendering and CESM model compilation. The container was rebuilt from scratch on Ubuntu 24.04, with native support for both processor architectures, modern packages, and a reproducible build system.

What multi-platform support means

A Docker image is compiled code, and compiled code is specific to a processor architecture. An image built for Intel/AMD (called “amd64” or “x86_64”) contains machine instructions that ARM processors cannot execute directly, and vice versa. When you run an amd64 image on an ARM Mac, Docker uses a technology called **QEMU emulation** to translate instructions on the fly. This works, but it is slow, uses more memory, and can trigger subtle bugs in complex software.

Multi-platform support means publishing a single image tag (like `latest`) that actually contains two architecture-specific images behind the scenes, organized in what Docker calls a **manifest list**. When someone runs `docker pull` , their Docker client automatically selects the correct image for their machine. An engineer on an Intel Linux server gets the amd64 image. An engineer on an M3 MacBook gets the arm64 image. Neither needs to know or care about the

other architecture. The image tag, the `docker run` command, and all the documentation stay identical.

The approach

Rather than cross-compiling the old image, the team rebuilt the entire stack on a new foundation. The original image compiled five scientific libraries (MPICH, HDF5, NetCDF-C, NetCDF-Fortran, PNetCDF) from Fortran and C source code inside the container, a process that took 30–45 minutes and was architecture-specific. The rebuild replaced all of this with pre-built binary packages from conda-forge, a community package repository that publishes builds for both amd64 and arm64. This reduced build time to about five minutes and eliminated the most complex source of architecture-dependent code.

The project also upgraded the CESM version from 2.2.0 to 2.2.2, trimmed the Python environment from approximately 400 packages to 35 direct dependencies, introduced reproducible lockfiles for both architectures, and created a three-tier automated test suite that validates the container on each platform.

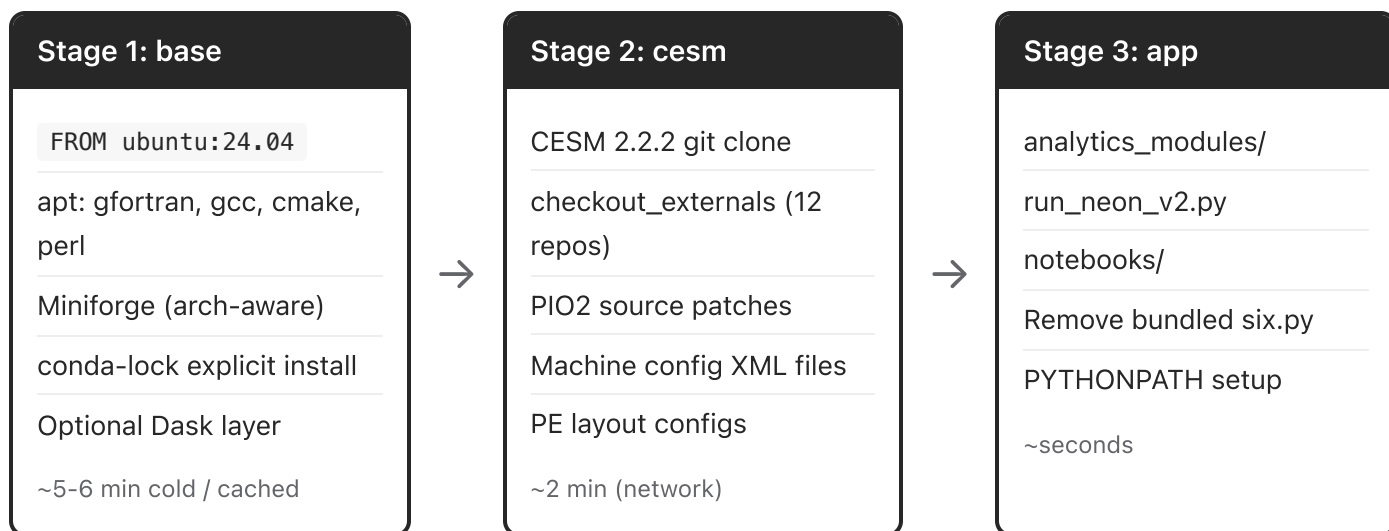
What changed for downstream consumers

Nothing, operationally. The image is published to the same GitHub Container Registry address. `docker pull` and `docker run` commands are unchanged. On Apple Silicon Macs, the container now runs natively instead of through emulation, so it is faster, more stable, and does not crash during heavy computation. On Intel/AMD machines, the container works as before, with updated package versions. The only user-visible difference is that Python is now 3.11 (up from 3.7), JupyterLab is version 4 (up from 2), and package versions have jumped by 4–6 years.

Section 2: Technical Detail

Dockerfile architecture

The Dockerfile (`./Dockerfile` , 174 lines) uses a three-stage linear build. All stages derive from the first; there is no separate builder stage because CESM's `case.build` workflow requires compilers at runtime.



Base image selection

Decision documented in [ADR-0001](#). Six alternatives were evaluated:

Option	Multi-arch	Size	EOL	Community fit
Ubuntu 24.04 (chosen)	amd64, arm64 +4	~29 MB	2029/2036	Pangeo, Jupyter Stacks
AlmaLinux 9	amd64, arm64	~30 MB	2032	CERN, Fermilab
Rocky Linux 9	amd64, arm64	~44 MB	2032	CIQ HPC
Debian 12	amd64, arm64	~47 MB	2028 LTS	Less common for HPC
jupyter/scipy-notebook	amd64, arm64	~1.5 GB	Follows Ubuntu	Jupyter community
condaforge/miniforge3	amd64, arm64	~148 MB	Follows Ubuntu	Pangeo pattern

Architecture detection

The build uses `$(uname -m)` for architecture detection rather than Buildx `TARGETARCH` / `TARGETPLATFORM` variables. Under QEMU emulation, `uname -m` reports the emulated architecture correctly. This is used in two places:

```
# Miniforge installer
wget ... "Miniforge3-Linux-$(uname -m).sh"

# Lockfile selection
ARCH=$(uname -m)
if [ "$ARCH" = "aarch64" ]; then LOCKFILE=...aarch64.lock;
else LOCKFILE=...64.lock; fi
```

CI integration

Single-job build in `.github/workflows/docker-publish.yml`:

```
steps:
  - docker/setup-qemu-action@v3      # ARM64 emulation
  - docker/setup-buildx-action@v3    # Multi-platform builder
  - docker/build-push-action@v6
    platforms: linux/amd64,linux/arm64
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

No matrix builds; both architectures built in a single Buildx invocation. QEMU handles arm64 on amd64 CI runners. Layer cache uses GitHub Actions cache backend.

CESM build compatibility fixes

Six issues resolved to make CESM 2.2.2 compile with modern conda-forge GCC 15.x and NetCDF-C 4.9+ on arm64:

Issue	Root cause	Fix
<code>rdtsc</code> asm error in GPTL	x86-only instruction	Removed <code>-DHAVE_NANOTIME -DBIT64</code>
MPI type mismatch	GCC 10+ strict checking	<code>-fallow-argument-mismatch</code>
BOZ hex literal error	GCC 10+ strict BOZ	<code>-fallow-invalid-boz</code>
<code>_FillValue</code> undeclared	NetCDF-C renamed macro	<code>-D_FillValue=NC_FillValue</code>
PIO2 filter API mismatch	NetCDF headers expose functions PIO2 doesn't implement	<code>sed</code> patches on PIO Fortran files
<code>cmake_minimum_required(3.0.2)</code>	cmake 4.x removed compat <3.5	Pin <code>cmake >=3.28,<4</code>

Test results

89/89 PASSED

94 seconds

 on native arm64

Tier 0: 63 smoke

Tier 1: 12 case

Tier 2: 14 build+analysis

Tier	Tests	Runtime	Coverage
0: Smoke	63	~4s	28 Python imports, CIME/CTSM modules, compilers, MPI, NetCDF libs, CESM install, env vars, dateutil/six regression
1: Case creation	12	~3s	query_config, create_newcase (I2000CIm50Sp/f19_g17), case.setup, xmlchange/xmlquery, run_neon_v2 CLI
2: Build + analysis	14	~85s	Full case.build (cesm.exe in 78s), NetCDF I/O, xarray workflows, cartopy rendering, Dask lazy I/O

Commit history

82d636a

Add ADRs for arm64 multi-arch container rebuild

4 ADRs: base image, build strategy, conda environment, distributed computing

- 1fd9f21

Rebuild container for multi-arch on Ubuntu 24.04

23 files, +15,767 lines. New Dockerfile, lockfiles, configs, docs
- 307d8e9

Add container validation test suite; pin Python to 3.11

89-test framework across 3 tiers. CIME needs Python <3.12
- 539a92b

Fix container config issues found by test suite

default_run_suffix, ctsm.download_utils, PYTHONPATH, USER env
- 9639015

Add XML::LibXML for CESM namelists

Perl module needed by CLM's build-namelist script
- 5b625dd

Fix CESM case.build on arm64 with modern toolchain

6 compat fixes: rdtsc, MPI types, BOZ, _FillValue, PIO2 filters, cmake pin

Image metrics

Metric	Value
Uncompressed size (arm64)	6.83 GB
Layer count	28
Dominant layers	conda env (~800 MB), CESM source tree (~2 GB), Ubuntu + apt (~550 MB)
Build time (cold, native arm64)	~8 min
Build time (code-only change)	~10 seconds
case.build (native arm64)	78 seconds

Known limitations

1. **No CI test integration yet.** Test suite is manual; adding tier0+tier1 to the GHA workflow is a natural follow-up.
2. **amd64 not locally tested.** Only arm64 exercised through the test suite. amd64 relies on CI build success.
3. **PIO2 patches are fragile.** `sed` macro renames will need updating if PIO2 or NetCDF-C versions change.
4. **Python pinned to 3.11.** CIME 2.2.x uses `import imp` removed in 3.12. CESM 3.x fixes this.
5. **No BuildKit cache mounts.** `--mount=type=cache` for conda packages could speed up cache-miss rebuilds.
6. **Large image size.** CESM source tree dominates; in-tree build model prevents separating build tools from runtime.